

返回到调用函数处^①。每一个 C 语言程序都必须有一个 main 函数。函数体由花括号 {} 括起来。本例中主函数内有两个语句,程序第 4 行是一个输出语句,printf 是 C 编译系统提供的函数库中的输出函数(详见第 4 章)。printf 函数中双撇号内的字符串 "This is a C program." 按原样输出。\\n 是换行符,即在输出 "This is a C program." 后,显示屏上的光标位置移到下一行的开头。这个光标位置称为输出的当前位置,即下一个输出的字符出现在此位置上。每个语句最后都有一个分号,表示语句结束。

在使用函数库中的输入输出函数时,编译系统要求程序提供有关此函数的信息(例如对输入输出函数的声明和宏的定义、全局量的定义等,这些以后会介绍的),程序第 1 行 "#include <stdio.h>" 的作用就是用来提供这些信息的。stdio.h 是系统提供的一个文件名,stdio 是 standard input & output 的缩写,文件后缀.h 的意思是头文件(header file),因为这些文件都是放在程序各文件模块的开头的。输入输出函数的相关信息已事先放在 stdio.h 文件中。现在,用 #include 指令把这些信息调入供使用。如果没有此 #include 指令,就不可能执行 printf 函数。关于编译预处理指令 #include,读者可先不必深究,只要记住:在程序中如要用到标准函数库中的输入输出函数,应该在本文件模块的开头加下面一行:

```
#include <stdio.h>
```

在以上程序各行的右侧,如果有 //,则表示从此处到本行结束是“注释”,用来对程序有关部分进行必要的说明。在写 C 程序时应当多用注释,以方便自己和别人理解程序各部分的作用。在程序进行预编译处理时将每个注释替换为一个空格,因此在编译时注释部分不产生目标代码,注释对运行不起作用。注释只是给人看的,而不是让计算机执行的。

 说明: C 语言允许用两种注释方式:

(1) 以 // 开始的单行注释。如上面介绍的注释。这种注释可以单独占一行,也可以出现在一行中其他内容的右侧。此种注释的范围从 // 开始,以换行符结束。也就是说这种注释不能跨行。如果注释内容一行内写不下,可以用多个单行注释,如下面两行是连续的注释行:

```
//如注释内容一行内写不下
```

```
//可以在下一行重新用“//”,然后继续写注释。
```

(2) 以 /* 开始,以 */ 结束的块式注释。这种注释可以包含多行内容。它可以单独占一行(在行开头以 /* 开始,行末以 */ 结束),也可以包含多行。编译系统在发现一个 /* 后,会开始找注释结束符 */,把二者间的内容作为注释。

但应注意的是在字符串中的 // 和 /* 都不作为注释的开始。而是作为字符串的一部分。如:

```
printf("//how do you do!\n");
```

^① C 99 建议把 main 函数指定为 int 型(整型),它要求函数带回一个整数值。在 main 函数中,在执行的最后设置一个 "return 0;" 语句。当主函数正常结束时,得到的函数值为 0;当执行 main 函数过程中出现异常或错误时,函数值为一个非 0 的整数。这个函数值是返回给调用 main 函数的操作系统的。程序员可以利用操作指令检查 main 函数的返回值,从而判断 main 函数是否已正常执行,并据此决定以后的操作。如果在程序中不写 "return 0;" 语句,有的 C 编译系统会在目标程序中自动加上这一语句,因此。主函数正常结束时,也能使函数值为 0。为使程序规范和可移植,希望读者写的程序一律将 main 函数指定为 int 型,并在 main 函数的最后加一个 "return 0;" 语句。

或

```
printf(" * how do you do! * /\n");
```

输出分别是:

```
//how do you do!
```

和

```
/* how do you do! */
```

注释可以用汉字或英文字符表示。

在C 89只允许用/*...*/形式的注释,而C++则允许用//形式的注释,//注释被称为“C++风格”的注释。但许多C编译系统在C 99之前就已支持这种方便的注释方法,C 99正式将//注释纳入C语言新标准。目前使用的一些编译系统(如Visual C++ 6.0, Turbo C++ 3.0和GCC)等都支持//单行注释。在本书的程序中,将利用//对程序的各部分作简要的说明。如果读者输入并运行这些程序,可不必包括这些注释内容。

【例 1.2】 求两个整数之和。

解题思路: 设置3个变量,a和b用来存放两个整数,sum用来存放和数。用赋值运算符“=”把相加的结果传送给sum。

编写程序:

```
#include <stdio.h>           //这是编译预处理指令
int main( )                  //定义主函数
{                             //函数开始
    int a,b,sum;             //本行是程序的声明部分,定义a,b,sum为整型变量
    a=123;                   //对变量a赋值
    b=456;                   //对变量b赋值
    sum=a+b;                 //进行a+b的运算,并把结果存放在变量sum中
    printf("sum is %d\n",sum); //输出结果
    return 0;                //使函数返回值为0
}                             //函数结束
```

运行结果:

```
sum is 579
```

然后换行,程序执行结束。

 **程序分析:** 本程序的作用是求两个整数a和b之和。第4行是声明部分,定义a,b和sum为整型(int)变量。第5,6行是两个赋值语句,使a和b的值分别为123和456。第7行使sum的值为a与b之和。第8行输出结果,这个printf函数圆括号内有两个参数。第一个参数是双撇号中的内容sum is %d\n,它是输出格式字符串,作用是输出用户希望输出的字符和输出的格式。其中sum is是用户希望输出的字符(这和例1.1是一样的),%d是指定的输出格式,d表示用“十进制整数”形式输出。圆括号内第二个参数sum表示要输出变量sum的值。在执行printf函数时,将sum变量的值(以十进制整数表示)取代双撇号中的%d。现在sum的值是579(即123与456之和),所以在输出时,十进制整数579取代

了%d(见图 1.1),\n 是换行符。

最后输出双撇号中的字符 sum is 579, 然后换行, 程序执行结束。

由于本程序正常运行和结束, 因此 main 函数的返回值应为 0。现在并没有去检查和利用这个函数值, 但是以后在某些时候会需要用到 main 函数值的。

【例 1.3】求两个整数中的较大者。

解题思路: 用一个函数来实现求两个整数中的较大者。在主函数中调用此函数并输出结果。

编写程序:

```
#include <stdio.h>
//主函数
int main()
{
    int max(int x,int y);
    int a,b,c;
    scanf("%d,%d",&a,&b);
    c=max(a,b);
    printf("max=%d\n",c);
    return 0;
}

//定义主函数
//主函数体开始
//对被调用函数 max 的声明
//定义变量 a,b,c
//输入变量 a 和 b 的值
//调用 max 函数,将得到的值赋给 c
//输出 c 的值
//返回函数值为 0
//主函数体结束

//求两个整数中的较大者的 max 函数
int max(int x,int y)
{
    int z;
    if(x>y)z=x;
    else z=y;
    return(z);
}
```

//定义 max 函数,函数值为整型,形式参数 x 和 y 为整型
//max 函数中的声明部分,定义本函数中用到的变量 z 为整型
//若 x>y 成立,将 x 的值赋给变量 z
//否则(即 x>y 不成立),将 y 的值赋给变量 z
//将 z 的值作为 max 函数值,返回到调用 max 函数的位置

运行结果:

```
8,5
max=8
```

在运行时,第 1 行输入 8 和 5,赋给变量 a 和 b,程序在第 2 行输出“max=8”。

程序分析: 本程序包括两个函数:①主函数 main;②被调用的函数 max。

max 函数的作用是将 x 和 y 中较大者的值赋给变量 z。第 18 行 return 语句将 z 的值作为 max 的函数值返回给调用 max 函数的函数(即主函数 main)。返回值是通过函数名 max 带回到 main 函数中去的(带回到程序第 8 行,main 函数调用 max 函数处)。

程序第 5 行是对被调用函数 max 的声明(declaration)。为什么要作这个函数声明呢?因为在主函数中要调用 max 函数(程序第 8 行“c=max(a,b);”),而 max 函数的定义却在 main 函数之后,对程序的编译是自上而下进行的,在对程序第 8 行进行编译时,编译系统无法知道 max 是什么,因而无法把它作为函数调用处理。为了使编译系统能识别 max 函数,

输出时用 sum 的值取代 %d

```
printf("sum is %d\n",sum);
```

图 1.1

就要在调用 max 函数之前用“int max(int x,int y);”对 max 函数进行“声明”,所谓声明,通俗地说就是告诉编译系统 max 是什么,以及它的有关信息。有关函数的声明详见第 7 章。

程序第 7 行 scanf 是输入函数的名字(scanf 和 printf 都是 C 的标准输入输出函数)。该 scanf 函数的作用是输入变量 a 和 b 的值。scanf 后面圆括号中包括两部分内容。一是双撇号中的内容,它指定输入的数据按什么格式输入。“%d”的含义是“以十进制整数形式”。二是输入的数据准备放到哪里,即赋给哪个变量。现在,scanf 函数中指定的是 a 和 b,在 a 和 b 的前面各有一个 &,在 C 语言中“&”是地址符,&a 的含义是“变量 a 的地址”,&b 是“变量 b 的地址”。执行 scanf 函数,从键盘读入两个整数,放到变量 a 和 b 的地址,然后把这两个整数分别赋给变量 a 和 b。

程序第 8 行用 max(a,b)调用 max 函数。在调用时将 a 和 b 作为 max 函数的参数(称为实际参数)的值分别传送给 max 函数中的参数 x 和 y(称为形式参数),然后执行 max 函数的函数体(程序第 14~19 行),使 max 函数中的变量 z 得到一个值(即 x 和 y 中大者的值),return(z)的作用是把 z 的值作为 max 函数值带回到程序第 8 行“=”的右侧(主函数调用 max 函数的位置),取代 max(a,b),然后把这个值赋给变量 c。

第 9 行用来输出结果。在执行 printf 函数时,对双撇号括起来的 max=%d\n 是这样处理的:将 max=原样输出,%d 由变量 c 的值取代,\n 的作用是换行。

 **注意:** 本例程序中两个函数都有 return 语句,请注意它们的异同。两个函数都定义为整型,都有函数值,都需要用 return 语句为函数指定返回值。但是 main 函数中的 return 语句指定的返回值一般为 0,而 max 函数的返回值是 max 函数中求出的两数中的最大值 z,只有通过 return 语句才能把求出的 z 值作为函数的值并返回调用它的 main 函数中(即程序第 8 行,并把此值赋给变量 c)。不要以为在 max 函数中求出最大值 z 后就会自动地作为函数值返回调用处,必须用 return 语句指定将哪个值作为函数值。也不要不加分析地在所有函数的最后都写上“return 0;”。

本例用到了函数调用、实际参数和形式参数等概念,只作了简单的解释。初学者对此可能不大理解,可以先不予深究,在学到以后有关章节时自然迎刃而解。在本章介绍此例子,主要是使读者对 C 程序的组成和形式有一个初步的了解。

1.4.2 C 语言程序的结构

通过以上几个程序例子,可以看到一个 C 语言程序的结构有以下特点:

(1) 一个程序由一个或多个源程序文件组成。一个规模较小的程序,往往只包括一个源程序文件,如例 1.1 和例 1.2 是一个源程序文件中只有一个函数(main 函数),例 1.3 中有两个函数,属于同一个源程序文件。

在一个源程序文件中可以包括 3 个部分:

① **预处理指令。**如 #include <stdio.h>(还有一些其他预处理指令,如 #define 等)。C 编译系统在对源程序进行“翻译”以前,先由一个预处理器(也称预处理程序、预编译器)对预处理指令进行预处理,对于 #include <stdio.h> 指令来说,就是将 stdio.h 头文件的内容读进来,取代 #include <stdio.h>。由预处理得到的结果与程序其他部分一起,组成一个完整的、可以用来编译的最后的源程序,然后由编译程序对该源程序正式进行编译,才得到目标程序。

② **全局声明**。即在函数之外进行的数据声明。例如可以把例 1.2 程序中的“int a,b,sum;”放到 main 函数的前面,这就是全局声明,在函数外面声明的变量称为全局变量。如果是在程序开头(定义函数之前)声明的变量,则在整个源程序文件范围内有效。在函数中声明的变量是局部变量,只在函数范围内有效。关于全局变量和局部变量的概念和用法见本书第 7 章。在本章的例题中没有用全局声明,只有在函数中定义的局部变量。

③ **函数定义**。如例 1.1、例 1.2 和例 1.3 中的 main 函数和例 1.3 中的 max 函数,要指定每个函数的功能。在调用这些函数时,会完成函数定义中指定的功能。

(2) **函数是 C 程序的主要组成部分**。程序的几乎全部工作都是由各个函数分别完成的,函数是 C 程序的基本单位,在设计良好的程序中,每个函数都用来实现一个或几个特定的功能。编写 C 程序的工作主要就是编写一个个函数。

一个 C 语言程序是由一个或多个函数组成的,其中必须包含一个 main 函数(且只能有一个 main 函数)。例 1.1 和例 1.2 中的程序只由一个 main 函数组成,例 1.3 程序由一个 main 函数和一个 max 函数组成,它们组成一个源程序文件,在进行编译时对整个源程序文件统一进行编译。

一个小程序只包含一个源程序文件,在一个源程序文件中包含若干个函数(其中有一个 main 函数)。当程序规模较大时,所包含的函数的数量较多,如果把所有的函数都放在同一个源程序文件中,则此文件显得太大,不便于编译和调试。为了便于调试和管理,可以使一个程序包含若干个源程序文件,每个源程序文件又包含若干个函数。一个源程序文件就是一个程序模块,即将一个程序分成若干个程序模块。

在进行编译时是以源程序文件为对象进行的。在分别对各源程序文件进行编译并得到相应的目标程序后,再将这些目标程序连接成为一个统一的二进制的可执行程序。

C 语言的这种特点使得容易实现程序的模块化。

在程序中被调用的函数,可以是系统提供的库函数(例如 printf 和 scanf 函数),也可以是由用户根据需要自己编制设计的函数(例如例 1.3 中的 max 函数)。C 的函数库十分丰富,ANSI C 建议提供了一百多个标准库函数,不同的 C 编译系统除了提供标准库函数外,还增加了其他一些专门的函数,如 Turbo C 提供了三百多个库函数。不同编译系统所提供的库函数个数和功能是不完全相同的。

(3) **一个函数包括两个部分**。

① **函数首部**。即函数的第 1 行,包括函数名、函数类型、函数属性、函数参数(形式参数)名、参数类型。

例如,例 1.3 中的 max 函数的首部为

int	max	(int	x,	int	y)
↓	↓	↓	↓	↓	↓
函数类型	函数名	函数参数类型	函数参数名	函数参数类型	函数参数名

一个函数名后面必须跟一对圆括号,括号内写函数的参数名及其类型。如果函数没有参数,可以在括号中写 void,也可以是空括号,如:

```
int main(void)
```

或

```
int main()
```

② **函数体**。即函数首部下面的花括号内的部分。如果在一个函数中包括有多层花括号,则最外层的一对花括号是函数体的范围。

函数体一般包括以下两部分。

- **声明部分**。声明部分包括:定义在本函数中所用到的变量,如例 1.3 中在 main 函数中定义变量“int a,b,c;”;对本函数所调用函数进行声明,如例 1.3 中在 main 函数中对 max 函数的声明“int max(int x,int y);”。
- **执行部分**。由若干个语句组成,指定在函数中所进行的操作。

在某些情况下也可以没有声明部分(例如例 1.1),甚至可以既无声明部分也无执行部分。如:

```
void dump ()  
{}
```

是一个空函数,什么也不做,但这是合法的。

(4) 程序总是从 main 函数开始执行的,而不论 main 函数在整个程序中的位置如何(main 函数可以放在程序最前头,也可以放在程序最后,或在一些函数之前、另一些函数之后)。

(5) 程序中要求计算机的操作是由函数中的 C 语句完成的。如赋值、输入输出数据的操作都是由相应的 C 语句实现的。

C 程序书写格式是比较自由的。一行内可以写几个语句,一个语句可以分写在多行上,但为清晰起见,习惯上每行只写一个语句。

(6) 在每个数据声明和语句的最后必须有一个分号。分号是 C 语句的必要组成部分。如:

```
c=a+b;
```

中的分号是不可缺少的。

(7) C 语言本身不提供输入输出语句。输入和输出的操作是由库函数 scanf 和 printf 等函数来完成的。C 语言对输入输出实行“函数化”。由于输入输出操作涉及具体的计算机设备,把输入输出操作由库函数实现,就可以使 C 语言本身的规模较小,编译程序简单,很容易在各种机器上实现,程序具有可移植性。

(8) 程序应当包含注释。一个好的、有使用价值的源程序都应当加上必要的注释,以增加程序的可读性。

1.5 运行 C 程序的步骤与方法

在 1.4 节中看到的用 C 语言编写的程序是源程序。计算机不能直接识别和执行用高级语言写的指令,必须用编译程序(也称编译器)把 C 源程序翻译成二进制形式的目标程序,然后再将该目标程序与系统的函数库以及其他目标程序连接起来,形成可执行的目标程序。

在编写好一个 C 源程序后,怎样上机进行编译和运行呢?一般要经过以下几个步骤: